

Planning

At the heart of a foundation model agent is the model responsible for solving a task. A task is defined by its goal and constraints. For example, one task is to schedule a two-week trip from San Francisco to India with a budget of \$5,000. The goal is the two-week trip. The constraint is the budget.

Complex tasks require planning. The output of the planning process is a plan, which is a roadmap outlining the steps needed to accomplish a task. Effective planning typically requires the model to understand the task, consider different options to achieve this task, and choose the most promising one.

If you’ve ever been in any planning meeting, you know that planning is hard. As an important computational problem, planning is well studied and would require several volumes to cover. I’ll only be able to cover the surface here.

Planning overview

Given a task, there are many possible ways to decompose it, but not all of them will lead to a successful outcome. Among the correct solutions, some are more efficient than others. Consider the query, “How many companies without revenue have raised at least \$1 billion?” There are many possible ways to solve this, but as an illustration, consider the two options:

1. Find all companies without revenue, then filter them by the amount raised.
2. Find all companies that have raised at least \$1 billion, then filter them by revenue.

The second option is more efficient. There are vastly more companies without revenue than companies that have raised \$1 billion. Given only these two options, an intelligent agent should choose option 2.

You can couple planning with execution in the same prompt. For example, you give the model a prompt, ask it to think step by step (such as with a chain-of-thought prompt), and then execute those steps all in one prompt. But what if the model comes up with a 1,000-step plan that doesn’t even accomplish the goal? Without oversight, an agent can run those steps for hours, wasting time and money on API calls, before you realize that it’s not going anywhere.

To avoid fruitless execution, *planning* should be decoupled from *execution*. You ask the agent to first generate a plan, and only after this plan is *validated* is it executed. The plan can be validated using heuristics. For example, one simple heuristic is to eliminate plans with invalid actions. If the generated plan requires a Google search and the agent doesn’t have access to Google Search, this plan is invalid. Another simple heuristic might be eliminating all plans with more than X steps. A plan can also be validated using AI judges. You can ask a model to evaluate whether the plan seems reasonable or how to improve it.

If the generated plan is evaluated to be bad, you can ask the planner to generate another plan. If the generated plan is good, execute it. If the plan consists of external tools, function calling will be invoked. Outputs from executing this plan will then again need to be evaluated. Note that the generated plan doesn’t have to be an

end-to-end plan for the whole task. It can be a small plan for a subtask. The whole process looks like [Figure 6-9](#).

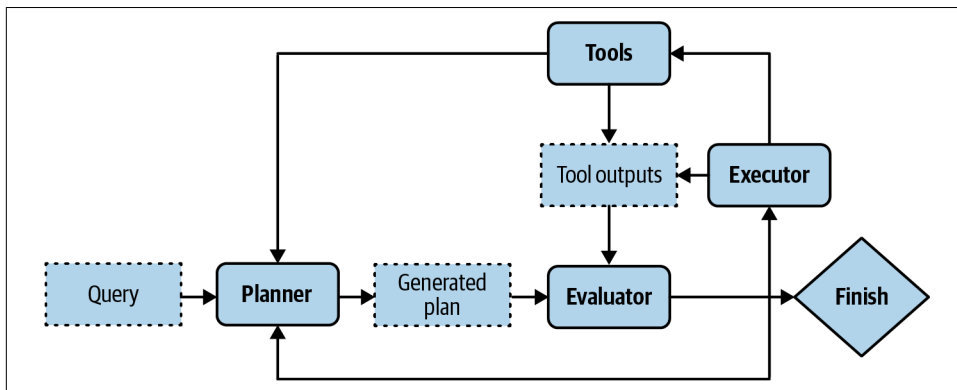


Figure 6-9. Decoupling planning and execution so that only validated plans are executed.

Your system now has three components: one to generate plans, one to validate plans, and another to execute plans. If you consider each component an agent, this is a multi-agent system.¹²

To speed up the process, instead of generating plans sequentially, you can generate several plans in parallel and ask the evaluator to pick the most promising one. This is another latency/cost trade-off, as generating multiple plans simultaneously will incur extra costs.

Planning requires understanding the intention behind a task: what’s the user trying to do with this query? An intent classifier is often used to help agents plan. As shown in [“Break Complex Tasks into Simpler Subtasks” on page 224](#), intent classification can be done using another prompt or a classification model trained for this task. The intent classification mechanism can be considered another agent in your multi-agent system.

Knowing the intent can help the agent pick the right tools. For example, for customer support, if the query is about billing, the agent might need access to a tool to retrieve a user’s recent payments. But if the query is about how to reset a password, the agent might need to access documentation retrieval.

¹² Because most agentic workflows are sufficiently complex to involve multiple components, most agents are multi-agent.



Some queries might be out of the scope of the agent. The intent classifier should be able to classify requests as IRRELEVANT so that the agent can politely reject those instead of wasting FLOPs coming up with impossible solutions.

So far, we’ve assumed that the agent automates all three stages: generating plans, validating plans, and executing plans. In reality, humans can be involved at any of those stages to aid with the process and mitigate risks. A human expert can provide a plan, validate a plan, or execute parts of a plan. For example, for complex tasks for which an agent has trouble generating the whole plan, a human expert can provide a high-level plan that the agent can expand upon. If a plan involves risky operations, such as updating a database or merging a code change, the system can ask for explicit human approval before executing or let humans execute these operations. To make this possible, you need to clearly define the level of automation an agent can have for each action.

To summarize, solving a task typically involves the following processes. Note that reflection isn’t mandatory for an agent, but it’ll significantly boost the agent’s performance:

1. *Plan generation*: come up with a plan for accomplishing this task. A plan is a sequence of manageable actions, so this process is also called task decomposition.
2. *Reflection and error correction*: evaluate the generated plan. If it’s a bad plan, generate a new one.
3. *Execution*: take the actions outlined in the generated plan. This often involves calling specific functions.
4. *Reflection and error correction*: upon receiving the action outcomes, evaluate these outcomes and determine whether the goal has been accomplished. Identify and correct mistakes. If the goal is not completed, generate a new plan.

You’ve already seen some techniques for plan generation and reflection in this book. When you ask a model to “think step by step”, you’re asking it to decompose a task. When you ask a model to “verify if your answer is correct”, you’re asking it to reflect.

Foundation models as planners

An open question is how well foundation models can plan. Many researchers believe that foundation models, at least those built on top of autoregressive language models, cannot. Meta’s Chief AI Scientist Yann LeCun states unequivocally that **autoregressive LLMs can’t plan** (2023). In the article “Can LLMs Really Reason and Plan?” **Kambhampati (2023)** argues that LLMs are great at extracting knowledge but not planning. Kambhampati suggests that the papers claiming planning abilities of LLMs confuse general planning knowledge extracted from the LLMs with executable plans.

“The plans that come out of LLMs may look reasonable to the lay user, and yet lead to execution time interactions and errors.”

However, while there is a lot of anecdotal evidence that LLMs are poor planners, it’s unclear whether it’s because we don’t know how to use LLMs the right way or because LLMs, fundamentally, can’t plan.

Planning, at its core, is a search problem. You search among different paths to the goal, predict the outcome (reward) of each path, and pick the path with the most promising outcome. Often, you might determine that no path exists that can take you to the goal.

Search often requires *backtracking*. For example, imagine you’re at a step where there are two possible actions: A and B. After taking action A, you enter a state that’s not promising, so you need to backtrack to the previous state to take action B.

Some people argue that an autoregressive model can only generate forward actions. It can’t backtrack to generate alternate actions. Because of this, they conclude that autoregressive models can’t plan. However, this isn’t necessarily true. After executing a path with action A, if the model determines that this path doesn’t make sense, it can revise the path using action B instead, effectively backtracking. The model can also always start over and choose another path.

It’s also possible that LLMs are poor planners because they aren’t given the toolings needed to plan. To plan, it’s necessary to know not only the available actions but also *the potential outcome of each action*. As a simple example, let’s say you want to walk up a mountain. Your potential actions are turn right, turn left, turn around, or go straight ahead. However, if turning right will cause you to fall off the cliff, you might not want to consider this action. In technical terms, an action takes you from one state to another, and it’s necessary to know the outcome state to determine whether to take an action.

This means it’s not sufficient to prompt a model to generate only a sequence of actions like what the popular chain-of-thought prompting technique does. The paper “Reasoning with Language Model is Planning with World Model” (Hao et al., 2023) argues that an LLM, by containing so much information about the world, is capable of predicting the outcome of each action. This LLM can incorporate this outcome prediction to generate coherent plans.

Even if AI can’t plan, it can still be a part of a planner. It might be possible to augment an LLM with a search tool and state tracking system to help it plan.

Foundation Model (FM) Versus Reinforcement Learning (RL) Planners

The *agent* is a core concept in RL, which is defined in [Wikipedia](#) as a field “concerned with how an intelligent agent ought to take actions in a dynamic environment in order to maximize the cumulative reward.”

RL agents and FM agents are similar in many ways. They are both characterized by their environments and possible actions. The main difference is in how their planners work. In an RL agent, the planner is trained by an RL algorithm. Training this RL planner can require a lot of time and resources. In an FM agent, the model is the planner. This model can be prompted or finetuned to improve its planning capabilities, and generally requires less time and fewer resources.

However, there’s nothing to prevent an FM agent from incorporating RL algorithms to improve its performance. I suspect that in the long run, FM agents and RL agents will merge.

Plan generation

The simplest way to turn a model into a plan generator is with prompt engineering. Imagine that you want to create an agent to help customers learn about products at Kitty Vogue. You give this agent access to three external tools: retrieve products by price, retrieve top products, and retrieve product information. Here’s an example of a prompt for plan generation. This prompt is for illustration purposes only. Production prompts are likely more complex:

SYSTEM PROMPT

Propose a plan to solve the task. You have access to 5 actions:

```
get_today_date()
fetch_top_products(start_date, end_date, num_products)
fetch_product_info(product_name)
generate_query(task_history, tool_output)
generate_response(query)
```

The plan must be a sequence of valid actions.

Examples

Task: "Tell me about Fruity Fedora"

Plan: [fetch_product_info, generate_query, generate_response]

Task: "What was the best selling product last week?"

Plan: [fetch_top_products, generate_query, generate_response]

Task: {USER INPUT}

Plan:

There are two things to note about this example:

- The plan format used here—a list of functions whose parameters are inferred by the agent—is just one of many ways to structure the agent control flow.
- The `generate_query` function takes in the task’s current history and the most recent tool outputs to generate a query to be fed into the response generator. The tool output at each step is added to the task’s history.

Given the user input “What’s the price of the best-selling product last week”, a generated plan might look like this:

```
1. get_time()
2. fetch_top_products()
3. fetch_product_info()
4. generate_query()
5. generate_response()
```

You might wonder, “What about the parameters needed for each function?” The exact parameters are hard to predict in advance since they are often extracted from the previous tool outputs. If the first step, `get_time()`, outputs “2030-09-13”, then the agent can reason that the parameters for the next step should be called with the following parameters:

```
retrieve_top_products(
    start_date="2030-09-07",
    end_date="2030-09-13",
    num_products=1
)
```

Often, there’s insufficient information to determine the exact parameter values for a function. For example, if a user asks, “What’s the average price of best-selling products?”, the answers to the following questions are unclear:

- How many best-selling products does the user want to look at?
- Does the user want the best-selling products last week, last month, or of all time?

This means that models frequently have to guess, and guesses can be wrong.

Because both the action sequence and the associated parameters are generated by AI models, they can be hallucinated. Hallucinations can cause the model to call an invalid function or call a valid function but with wrong parameters. Techniques for improving a model’s performance in general can be used to improve a model’s planning capabilities.